ATLAS ROOM RESERVATION SYSTEM

# Library Seat & Resource Reservation with Priority Queue

Complete Technical Architecture, Data Flow, Priority Formula & Viva Guide

| | |
|---|---|
| **Project Domain:** | Library & Academic Resource Scheduling System |
| **Core Algorithm:** | Weighted Dynamic Priority Queue (Preemption & Tie-Breaking) |
| **Technical Stack:** | HTML5, CSS3 (Custom Design System), JavaScript (ES2017 Async/Await), JSON Server, REST API |
| **Database:** | db.json (Users, Resources, Reservations, Exam Windows) |
| **Target Audience:** | College Evaluators, External Viva Examiners, Academic Project Defense |
| **Documentation Date:** | August 2026 |

**Executive Summary:** This project replaces traditional First-Come-First-Serve (FCFS) library seat reservation with a fair, automated Priority Queue. Final-year/thesis students under active exam proximity receive higher priority during reservation conflicts, preempting ('bumping') lower-priority bookings or queuing them in a 'waiting' state, while ensuring transparent cancellation promotions and admin oversight.

## Table of Contents

**Part 1:** Project Overview & Problem Definition
**Part 2:** Complete System Architecture
**Part 3:** Codebase File Structure & Module Map
**Part 4:** Database Schema & Entity Relationships
**Part 5:** REST API & JSON Server Operations
**Part 6:** User Signup End-to-End Flow
**Part 7:** Login & Session Management
**Part 8:** Authentication & Authorization Guards
**Part 9:** Resource Browsing & Filtering Mechanism
**Part 10:** Priority Score Mathematical Formulation
**Part 11:** Booking Creation & Conflict Detection
**Part 12:** Priority Queue Preemption (Bumping/Waiting)
**Part 13:** Automatic Queue Promotion Logic
**Part 14:** My Bookings Dashboard & Cancellation
**Part 15:** Profile & Live Data Verification
**Part 16:** Admin Panel Complete Workflow
**Part 17:** Admin Reservation Oversight
**Part 18:** Admin Force Confirm Protection
**Part 19:** No-Show Tracking & Strike Penalty
**Part 20:** Exam Window Management & Proximity
**Part 21:** Logout & Session Destruction
**Part 22:** End-to-End Unified System Lifecycle
**Part 23:** Client-Server Persistence Model
**Part 24:** Complete Reservation Status Lifecycle
**Part 25:** JavaScript Curriculum & Syllabus Audit
**Part 26:** Top 30 Comprehensive Viva Questions
**Part 27:** One-Minute Natural Viva Speech
**Part 28:** Step-by-Step Live Demo Sequence

## Part 1: Project Overview

**The Problem:** In educational institutions, library study rooms, carrels, and computing terminals face extreme congestion during exam weeks. Conventional reservation systems rely on pure First-Come-First-Serve (FCFS). This allows junior students or casual visitors to hoard critical study resources weeks in advance, leaving final-year students, thesis researchers, and students with impending exams without access.

**The Solution:** A Priority Queue Reservation System that computes a dynamic priority score for each reservation attempt based on *User Role (50%)*, *Reliability History (30%)*, and *Exam Proximity (20%)*. When time slots conflict, higher-priority students preempt lower-priority bookings.

**Target Users & Flows:**

- **Students:** Sign up with academic course/semester, log in, browse resources, submit reservations, monitor queue status, cancel upcoming sessions.
- **Faculty / Researchers:** Reserve study rooms with specific faculty priority tier.
- **Admin / Librarians:** Oversee all bookings, force-confirm with overlap protection, mark no-shows with penalty deductions, manage resources, and configure exam windows.

**Core Technology Roles:** HTML5 for semantic page layout, CSS3 for the Atlas Room design system, vanilla JavaScript (ES2017 async/await) for client-side state and priority computation, LocalStorage for session token storage, Fetch API for asynchronous REST communication, and JSON Server with db.json for persistent data storage.

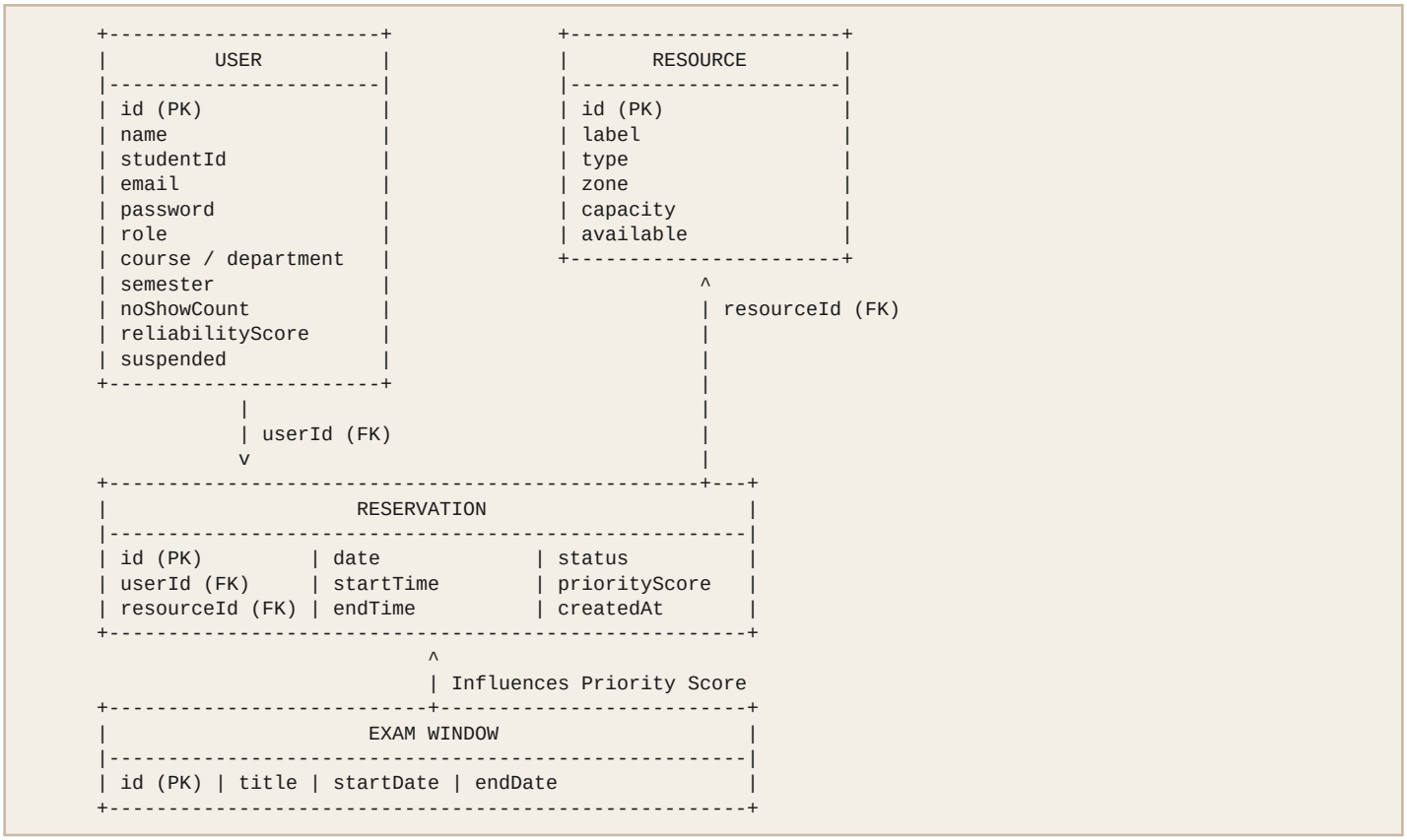
## Part 2: Complete System Architecture

The system operates as a single-tier client architecture communicating asynchronously with an API server backed by a JSON database:

## Part 3: File Structure & Module Map

| File Path | Type / Responsibility | Key Functions & APIs |
|---|---|---|
| frontend/index.html | Landing Page & Portal | Public navigation, Atlas Room showcase, links to auth. |
| frontend/sign_up.html | Student Registration | Validates course/semester, `GET /users` (uniqueness), `POST /users`. |
| frontend/login.html | User Authentication | `GET /users`, validates password, stores `currentUser` in localStorage. |
| frontend/booking.html | Resource Reservation | Loads `priority.js`, calls `GET /resources`, `POST /reservations`. |
| frontend/myBooking.html | Student Dashboard | `GET /reservations`, `PATCH /reservations/:id` (cancel + promotion). |
| frontend/profile.html | Member Profile Dashboard | `GET /users/:id` live sync, priority score display, logout handler. |
| frontend/admin.html | 5-Tab Admin SPA | Auth guard (`role === 'admin'`), loads `adminNav.js` and `adminPanel.js`. |
| frontend/js/priority.js | Shared Priority Core | `calculatePriorityScore`, `resolveBookingConflict`, `promoteWaitingReservation`. |
| frontend/js/adminPanel.js | Admin Controller | CRUD for resources, examWindows, reservations, user suspension. |
| frontend/js/adminNav.js | Admin Tab Switching | Manages `data-view` visibility and `adminview:shown` events. |
| frontend/css/app.css | Shared Application Design | Design tokens, typography, grid layouts, status pill badges. |
| db.json | JSON Database Store | Contains 4 collections: `users`, `resources`, `reservations`, `examWindows`. |

## Part 4: Database Schema & Entity Relationships



## Part 5: REST API & JSON Server Operations

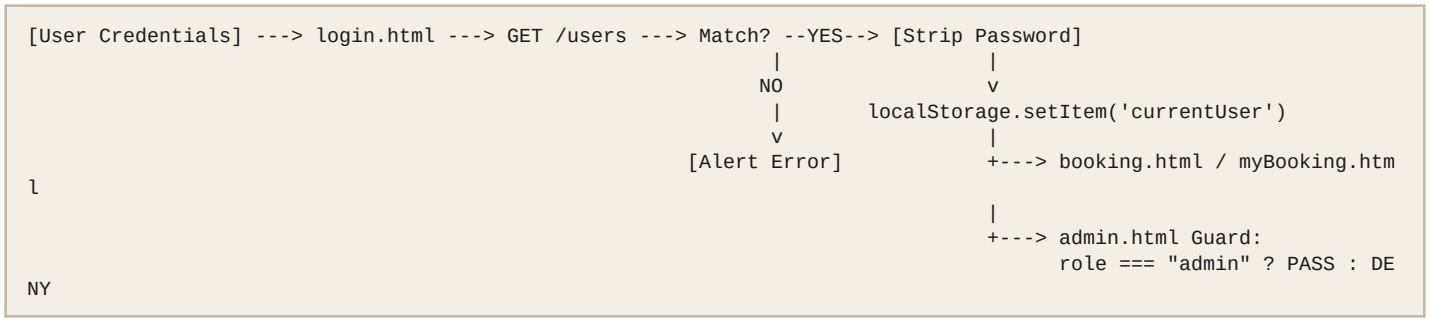
| Method | Endpoint | Used By | Purpose |
|---|---|---|---|
| GET | /users | login, sign_up, admin | Authenticate user / check email uniqueness / search users. |
| GET | /users/:id | booking, profile, admin | Fetch live suspension state, profile metrics, reliability. |
| POST | /users | sign_up.html | Create new registered student record. |
| PATCH | /users/:id | adminPanel.js | Update suspension status, increment noShowCount, update reliability. |
| GET | /resources | booking, myBooking, admin | Load all rooms, carrels, workstations, and capacities. |
| POST | /resources | adminPanel.js | Admin creates a new library resource. |
| PATCH | /resources/:id | adminPanel.js | Admin updates label, type, zone, or capacity. |
| DELETE | /resources/:id | adminPanel.js | Admin deletes a library resource. |
| GET | /reservations | booking, myBooking, admin | Load reservations for conflict detection and dashboard. |
| POST | /reservations | booking.html | Create a new confirmed or waiting reservation. |
| PATCH | /reservations/:id | priority.js, myBooking | Update status: bumped, cancelled, confirmed, no_show. |
| DELETE | /reservations/:id | adminPanel.js | Admin permanently removes a reservation record. |
| GET | /examWindows | priority.js, admin | Load active exam windows for proximity calculation. |
| POST | /examWindows | adminPanel.js | Admin adds an upcoming examination date range. |
| DELETE | /examWindows/:id | adminPanel.js | Admin deletes an exam window. |

## Part 6, 7 & 8: Authentication, Signup & Session Lifecycle

**1. Signup Flow:** User fills in Name, Student ID, Email, Password, Course (B.Tech/BSc/BBA/M.Tech/MSc/MBA/PhD), and Semester. The form dynamically bounds semester options to the course duration. JavaScript sends `GET /users` to verify email uniqueness. On success, `POST /users` creates the account with `noShowCount: 0, reliabilityScore: 100, suspended: false`.

**2. Login Flow:** `login.html` collects Email and Password, performs `GET /users`, and finds the matching object. Before saving, `delete user.password` strips the password for security, and `localStorage.setItem('currentUser', JSON.stringify(user))` writes the active session token.

**3. Authorization Guards:** Every protected page checks `localStorage.getItem('currentUser')`. If null, redirects to `login.html`. `admin.html` enforces a second guard: `currentUser.role === 'admin'`. Unauthorized visitors receive an access alert and are redirected to `index.html`.



## Part 9 & 10: Priority Score Mathematical Formulation

The priority calculation in `priority.js` uses a weighted composite formula producing a score between 0 and 100:

**FINAL PRIORITY FORMULA:**
**PriorityScore = Math.round( (RoleScore × 0.50) + (ReliabilityScore × 0.30) + (ExamProximityScore × 0.20) )**
*Result is strictly clamped between 0 and 100 points.*

**Component Breakdown:**

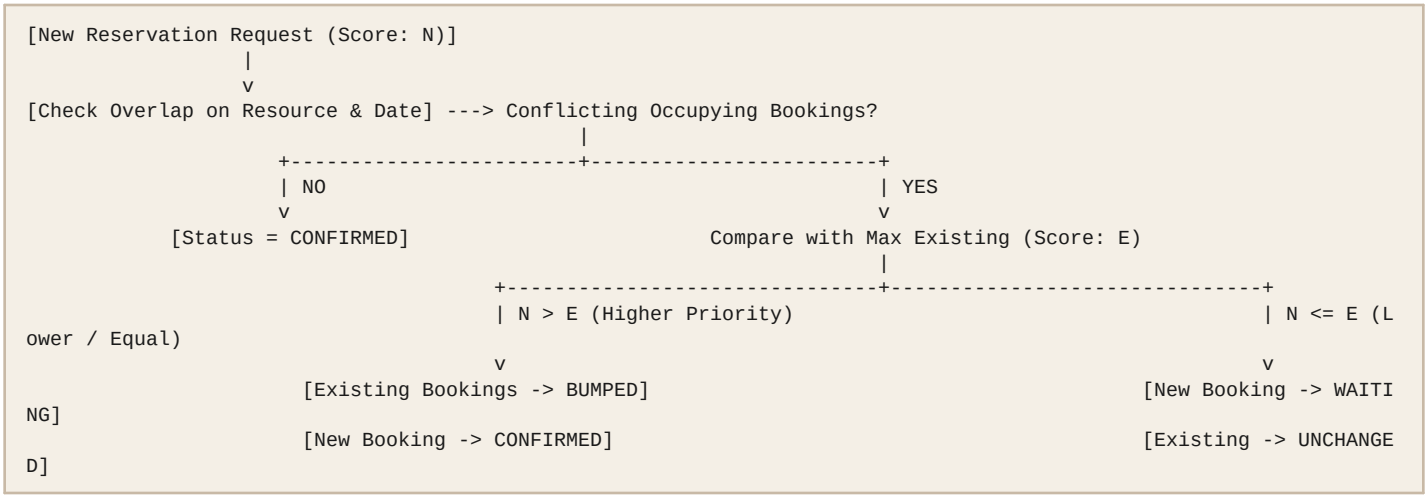
1. **Role Score (Weight: 50%):** Faculty = 50, Non-student = 30. For students, `isFinalYear(user)` checks if `semester >= maxDuration - 1` (e.g. B.Tech sem 7-8). Final-year student = 100, Senior student (`semester > midpoint`) = 75, Junior student = 60.
2. **Reliability Score (Weight: 30%):** Uses `user.reliabilityScore` directly if recorded, otherwise defaults to `100 - min(noShowCount, 6) × 15`. Deducts 15 points per recorded no-show strike.
3. **Exam Proximity Score (Weight: 20%):** Awarded ONLY to final-year students whose booking date falls within an active `/examWindows` period. Score = `Math.max(0, 100 - daysRemaining × 8)`. Non-final-year students receive 0.

**Numerical Example:** Student Priya (B.Tech Semester 8, Final Year $\rightarrow$ Role = 100), 0 No-shows (Reliability = 100), Booking 2 days before Mid-Term Exam (Exam Score = $100 - (2 \times 8) = 84$):
$$\text{Score} = (100 \times 0.50) + (100 \times 0.30) + (84 \times 0.20) = 50 + 30 + 16.8 = 96.8 \rightarrow \mathbf{97}\text{ points}.$$

## Part 11 & 12: Conflict Detection & Preemption (Bumping/Waiting)

When a user submits a booking, `booking.html` executes the following conflict resolution protocol:

- **Time Overlap Formula:** A conflict exists if another reservation matches `resourceId`, `date`, and satisfies `(startTime < other.endTime && endTime > other.startTime)` while having an occupying status (`confirmed` or `pending`).
- **Preemption (Bump):** If conflicting reservations exist, `resolveBookingConflict()` finds the highest existing priority score. If `newScore > existingScore`, all conflicting bookings are updated via `PATCH` to `status: 'bumped'`, and the new booking is created as `status: 'confirmed'`.
- **Queueing (Waiting):** If `newScore <= existingScore`, the new booking is assigned `status: 'waiting'`. Existing confirmed reservations are completely undisturbed.




## Part 13 & 14: Promotion Logic & My Bookings Dashboard

**Queue Promotion Protocol:** When any confirmed booking is cancelled (either by the student in `myBooking.html` or by an administrator in `admin.html`), `promoteWaitingReservation(resourceId, date, startTime, endTime)` automatically searches for all overlapping reservations with status `waiting` or `bumped`. It sorts candidates by `priorityScore DESC` and uses `createdAt ASC` (earlier timestamp) as a tie-breaker. The top candidate is immediately promoted via `PATCH` to `status: 'confirmed'`.

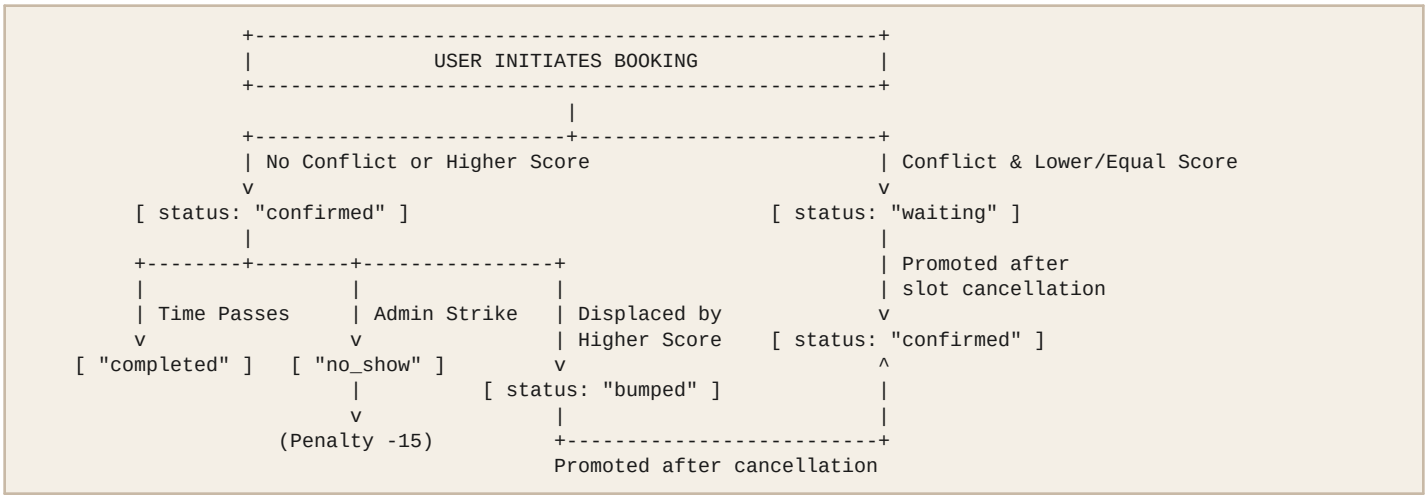
**My Bookings Interface:** Displays all reservations belonging to `currentUser.id`. Maps statuses to formatted badges (`Confirmed`, `Waiting`, `Bumped`, `Cancelled`, `Completed`, `No-show`) and calculates temporal status (`Upcoming`, `Active`, `Completed`). The 'Cancel Booking' action is enabled only for cancelable Upcoming slots.

## Part 15 to 21: Admin Control Panel & Management Features

The Admin panel (`admin.html`) provides centralized library resource oversight across 5 views:

1. **Dashboard:** Computes live metrics: total reservations today, active sessions occurring right now (`isHappeningNow`), pending count, and today's no-shows. Automatically refreshes on tab activation.
2. **Reservation Oversight:** Unified table with multi-criteria filters (status, date, user, resource). Includes action triggers: Force Confirm, Mark No-show, Cancel, and Delete.
3. **Force Confirm Protection:** Before confirming any waiting or pending reservation, `updateReservationStatus()` verifies whether another confirmed/pending booking currently occupies that slot. If occupied, confirmation is blocked with an alert.
4. **No-Show Management:** Admin clicks 'Mark No-show' on an unattended confirmed booking. The system updates the reservation to `status: 'no_show'`, increments `user.noShowCount`, and decrements `user.reliabilityScore` by 15.
5. **Resource & Exam Window CRUD:** Full create/read/update/delete management for library resources (labels, types, zones, capacities) and academic exam windows.

## Part 22 & 24: Complete Reservation Status Lifecycle




## Part 25: JavaScript Curriculum & Syllabus Audit

The project is written strictly in vanilla JavaScript adhering to university curriculum concepts:

- **Standard Concepts:** `let`/`const`, data types, arrays, objects, `if`/`else`, `for` loops, functions, arrow functions, `forEach`, `filter`, `sort`, `find`, template literals, DOM manipulation, form events, `localStorage`, `JSON.parse`/`stringify`, `fetch` API, `async`/`await`, Promises, REST operations.
- **Modern Utilities:** `Promise.all()` (parallel data loading), ternary operator, `Math` functions (`round`, `max`, `min`), `Date` arithmetic.
- **Out-of-Scope Verified Absent:** No React, Angular, Vue, Classes, ES Modules (`import`/`export`), MongoDB, Express app code, or external frameworks.

# Part 26: Top 30 Comprehensive Viva Questions & Answers

**1. What problem does this project solve?**

It eliminates unfair hoarding of library seats by replacing First-Come-First-Serve with a Priority Queue that prioritizes final-year and thesis students with upcoming exams.

**2. How is the priority score calculated?**

Using a weighted formula: Role Score (50%) + Reliability Score (30%) + Exam Proximity Score (20%), clamped between 0 and 100 points.

**3. What happens if two users have the exact same priority score?**

Equal priority does NOT bump an existing booking. The new booking goes to the waiting queue, and timestamp (createdAt) acts as a tie-breaker during promotion.

**4. What is 'Bumping'?**

Preemption where a new higher-priority booking automatically changes an existing lower-priority confirmed booking to 'bumped' status and takes the slot.

**5. What is 'Waiting' status?**

A queued state assigned to a reservation attempt when the requested slot is occupied by a booking with equal or higher priority.

**6. How does queue promotion work?**

When a confirmed booking is cancelled, the system finds all overlapping waiting/bumped bookings, sorts by priorityScore desc and createdAt asc, and promotes the winner to confirmed.

**7. Why is localStorage used?**

To store the logged-in user's session token ('currentUser') across page reloads without requiring complex session servers.

**8. Why is the password stripped before storing currentUser?**

Security best practice: `delete exist.password` ensures sensitive credentials are never retained in browser storage.

**9. What is JSON Server?**

A lightweight development tool that creates a full fake REST API server backed by a simple db.json file.

**10. What is db.json?**

The single-source JSON database containing users, resources, reservations, and examWindows collections.

**11. What is a REST API?**

Representational State Transfer: an architectural style for client-server communication using standard HTTP methods (GET, POST, PATCH, DELETE).

**12. What is the difference between POST and PATCH?**

POST creates an entirely new record; PATCH updates only specific modified fields of an existing record.

**13. How does the Fetch API work?**

It sends asynchronous HTTP network requests from the browser to the backend and returns a Promise resolving to the server's response.

**14. How do you detect overlapping booking times?**

Using the mathematical interval formula: `(startTime < existing.endTime && endTime > existing.startTime)` on the same resource and date.

**15. Which reservation statuses occupy a slot?**

Only 'confirmed' and 'pending' reservations occupy slots. 'waiting', 'bumped', 'cancelled', 'completed', and 'no_show' do not block bookings.

**16. How does a No-Show affect a student?**

Marking a no-show increments user.noShowCount and deducts 15 points from their reliability score, lowering their priority for future reservations.

### 17. What is an Exam Window?

A date range defined in db.json representing mid-terms or finals. Final-year students booking inside an active window get an exam proximity score boost.

### 18. How does admin authentication work?

admin.html checks `currentUser.role === 'admin'`. If false or missing, it blocks access and redirects to index.html/login.html.

### 19. What is Admin Force Confirm?

An administrative override to confirm a waiting booking. It includes conflict protection to prevent accidental double-bookings.

### 20. Why do we fetch live user data on the profile page?

To ensure dynamic updates (such as admin suspension or no-show strike penalties) are reflected immediately rather than relying on stale localStorage data.

### 21. How does resource filtering work?

JavaScript filters the resources array by matching dropdown values against resource.type and resource.zone properties.

### 22. Why use async/await in JavaScript?

It provides clean, synchronous-looking syntax for handling asynchronous Promises returned by the Fetch API.

### 23. What is DOM manipulation?

Using JavaScript APIs like `document.getElementById` and `innerHTML` to dynamically update page content without full page reloads.

### 24. Why is priority.js shared across pages?

It encapsulates the single source of truth for priority calculation, conflict resolution, and promotion, ensuring consistency between student booking and admin views.

### 25. How does My Bookings identify which reservations to show?

It filters the full reservations array using `String(reservation.userId) === String(currentUser.id)`.

### 26. What happens if a suspended user tries to book?

booking.html performs a live `GET /users/:id` check. If `user.suspended === true`, it alerts the user and aborts the booking.

### 27. How does the tie-breaker work during promotion?

If two waiting users have the exact same priority score, the earlier creation timestamp (`createdAt`) wins the promotion.

### 28. What happens to a bumped booking if the preemptor cancels?

The bumped booking is in the promotion candidate pool and will be automatically promoted back to 'confirmed' if it has the highest priority score.

### 29. Does the frontend directly modify db.json?

No. The frontend communicates exclusively via HTTP requests (GET, POST, PATCH, DELETE) to JSON Server, which handles disk persistence.

### 30. Is this system suitable for deployment?

Yes, the frontend and priority logic are production-ready and can be coupled with Node.js/Express and MongoDB for enterprise deployment.

## Part 27: One-Minute Viva Pitch

**Student Viva Speech (Memorize & Speak):**

"Good morning, Sir/Madam. My project is the **Library Seat & Resource Reservation with Priority Queue**.

Traditional library reservation systems operate on First-Come-First-Serve, which leads to congestion and unfairness when junior students hoard study spaces while final-year students preparing for thesis defenses or exams cannot find seats. My system solves this by introducing an automated, dynamic **Priority Queue**.

Every reservation computes a priority score out of 100 based on three weighted factors: **User Role (50%)** giving priority to final-year students, **Reliability History (30%)** which penalizes repeated no-shows, and **Exam Proximity (20%)** during active exam windows.

When a conflict occurs, a higher-priority student automatically **preempts or 'bumps'** a lower-priority booking, while equal or lower-priority requests enter a **waiting queue**. If a confirmed session is cancelled, the system automatically **promotes** the highest-priority waiting reservation.

The project is implemented with vanilla HTML5, CSS3, and JavaScript using asynchronous REST APIs with JSON Server and db.json, complete with an administrative dashboard for oversight, conflict-guarded force confirmation, and strike tracking."

## Part 28: Live Examiner Demo Sequence

1. **Login as Junior Student:** Log in as spider@gmail.com (B.Tech Sem 3, Junior -> Priority ~60).
2. **Create Baseline Booking:** Go to Reserve a Seat, select 'Individual Carrel C1' on tomorrow's date for 10:00 - 12:00 -> Status becomes **Confirmed**.
3. **Demonstrate Preemption (Bump):** Log out and log in as cap@gmail.com (B.Tech Sem 8, Final Year -> Priority ~80). Book the exact same Carrel and time slot -> Cap gets **Confirmed**; Spider's booking is automatically **Bumped**.
4. **Demonstrate Waiting Queue:** Log in as thor@gmail.com (B.Tech Sem 5 -> Priority ~73). Attempt to book the same slot -> Score (73) <= Cap (80) -> Thor receives **Waiting** status.
5. **Demonstrate Queue Promotion:** Log back in as Cap, open 'My Bookings', and click 'Cancel Booking'. Thor (Score 73 > Spider 60) is automatically **Promoted to Confirmed**.
6. **Demonstrate Admin Oversight:** Log in as Admin (admin@university.edu / password123). Open admin.html.
7. **Demonstrate No-Show Strike:** In the Reservations tab, click 'Mark No-show' on a confirmed booking -> Shows penalty applied to user reliability.
8. **Demonstrate Force Confirm Protection:** Try to Force Confirm an overlapping booking -> System blocks double-booking.
9. **Demonstrate Exam Windows:** Open Settings tab, show active exam windows that drive the 20% proximity boost.